HJM Bermudan Swaptions Pricer — Technical Report

Oct 1, 2026 · @Yassine

Overview

This repository is a from-scratch, dependency-light (NumPy only) one-factor HJM pricer for Bermudan coterminal swaptions, calibrated to European swaption quotes and cross-checked by two independent numerical engines.

- No PDF, slide deck or README exists in the repository; this report is reconstructed from the source and from running `run_example.py` directly.
- Package `hjm1f/`: `curve.py`, `hjm_1f.py`, `instruments.py`, `market_quotes.py`, `calibration_1f.py`, `pde_1f.py`, `mc_1f.py`, `solvers.py`.
- Entry points: `run_example.py` (script) and `Main.ipynb` (notebook version, plus sensitivity/convergence studies).

The state variable and bond pricing formula

The model drives a single driftless Gaussian state variable rather than the short rate directly:

$$X_t = \int_0^t \sigma(u)\,e^{\lambda u}\,dW_u, \qquad dX_t = \eta(t)\,dW_t,\quad \eta(t) = \sigma(t)\,e^{\lambda t}$$

σ(t) is piecewise-constant (one bucket per Bermudan exercise date, `PiecewiseConstantSigma`) and λ is a constant mean-reversion parameter. Because X_t has no drift, the pricing PDE carries no first-derivative term — this is algebraically a one-factor Hull-White / extended-Vasicek model, written in HJM state-variable notation and matched exactly to today's curve.

The discount bond price is exponential-affine in X_t:

$$P(t,T) = \frac{P(0,T)}{P(0,t)}\,\exp\!\big(D(t,T)\,(c(t,T)-X_t)\big)$$

with D(t,T) = B_λ(T) − B_λ(t), B_λ(t) = (1−e^(−λt))/λ, and c(t,T) = ½(B_λ(T)+B_λ(t))·α(t) − γ(t), where α(t)=Var[X_t] and γ(t) is a second deterministic integral — both closed-form and evaluated piecewise (`HJM1FModel.alpha` / `.gamma`). The short rate is recovered consistently by taking T→t:

```
r_t = f(0,t) - e^{-\lambda t}\big(B_\lambda(t)\,\alpha(t) - \gamma(t) - X_t\big)
```

Discount curve

`ZeroCurve` (`curve.py`) takes market discount factors P(0,T) directly as input at a set of pillar times — the project deliberately does not bootstrap a curve from deposits or swaps. Interpolation is linear in log-discounts (keeps discount factors positive), with flat extrapolation outside the pillar range. The instantaneous forward rate f(0,t), needed by the short-rate formula above, is obtained by a symmetric finite difference on log P(0,t) with step h=1e-5.

Market quotes and European pricing

`market_quotes.py` converts a normal-vol (Bachelier) quote into a market price: it computes the single-curve forward swap rate S(0) = (P(0,T_expiry)−P(0,T_N))/Annuity and applies the standard Bachelier payer/receiver formula. This is the calibration target.

The European coterminal swaption is priced by Jamshidian decomposition (`HJM1FModel.european_swaption_price_jamshidian`):

1. Root-find x* (the project's own Brent solver, `solvers.py`) such that the underlying swap's value at expiry is zero.
2. Each remaining coupon becomes a strike on a zero-coupon bond P(expiry, t_i, x*); the swaption price sums the corresponding bond-put values, priced by a closed-form formula using total bond volatility |D(expiry,maturity)|·√α(expiry).

The receiver branch is not fully specialized — the code returns the same expression for both `payer=True/False` — so pricing is effectively payer-only in practice.

Calibration: sequential bootstrap

`CoterminalBootstrap1F` (`calibration_1f.py`) assigns one σ bucket per Bermudan exercise date and calibrates sequentially in increasing expiry order. For each instrument, it converts the market normal-vol quote to a price via Bachelier, then solves a 1-D root (Brent, bounds [1e-6, 0.20] by default) for that bucket's σ so the Jamshidian model price matches the market price exactly, holding all earlier buckets fixed.

Because each step is a single monotone scalar root-find rather than a joint optimization, the fit is exact by construction — there is no smile, no cap/floor cross-calibration, and no global optimizer involved.

Bermudan engine 1: finite-difference PDE

`BermudanPDE1F` (`pde_1f.py`) solves the pricing PDE backward on a grid in X_t. The spatial range is ±5 standard deviations of X at the final exercise date, [−5√α(T_max), 5√α(T_max)] (`PDEGrid.around_factor_std`), since X_t is Gaussian; defaults are 301 spatial points and 500 time steps. The time grid is built so every Bermudan exercise date falls exactly on a grid point, avoiding interpolation error at exercise.

Each backward step is a fully-implicit finite-difference scheme — unconditionally stable — solved as a tridiagonal system via a hand-written Thomas algorithm. Because X_t is driftless, the PDE has no advection term; the local short rate r(t,x) enters only through the diagonal (discounting). Boundary values are "conservative": they inherit the continuation value from the next time slice, floored by the best remaining intrinsic value evaluated at the boundary — a documented fix over an earlier version that used an incorrect swap-value proxy as a Dirichlet boundary. At each exercise date the grid value is maxed against immediate intrinsic value (a standard Bellman step); the final price is read off the grid at X_0=0.

Bermudan engine 2: Longstaff–Schwartz Monte Carlo

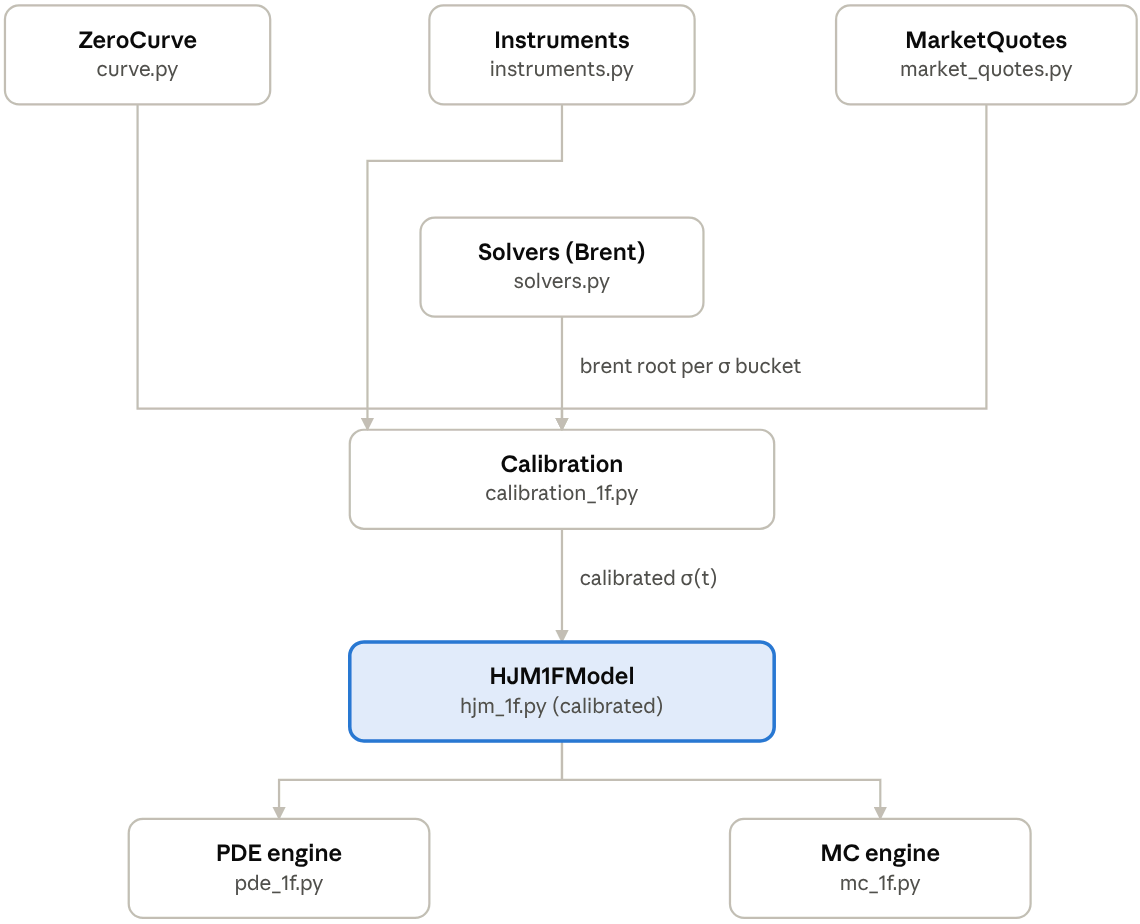
`BermudanLSMC1F` (`mc_1f.py`) is explicitly documented in the code as "not the main engine... included mainly as a rough cross-check against the PDE price." Since X_t is a pure Gaussian martingale, paths are simulated only at the exercise dates, using independent increments of size √(α(T_j)−α(T_{j-1})) — no path-discretization error.

The engine runs standard Longstaff–Schwartz backward regression with basis [1, X, X²] on in-the-money paths only. Continuation value is discounted from each exercise date to the currently assumed stopping time using the local short rate as a flat proxy, rather than a full stochastic discount-factor integral — an approximation acceptable at this short horizon but worth noting. The realized cashflow on each path is finally discounted to today using r_0 = short_rate(0,0).

Code architecture

Eight modules turn a discount curve into two independent Bermudan prices

hjm1f package call graph, driven by run_example.py



run_example.py wires these modules together, then compares the PDE and LSMC Bermudan prices.

hjm1f module call graph, 8 nodes

Worked example and results

`run_example.py` sets up a 10-year discount curve (pillars 0–10y), a Bermudan coterminal payer swaption exercisable at 1y/2y/3y into a swap terminating at 7y (annual coupons, 3% strike, λ=0.03), calibrated to three European coterminal swaptions at normal vols 80/85/90bp. Running the script directly (verified for this report) gives an exact bootstrap fit:

Exercise	Normal vol	Calibrated σ	Market price	Model price
1y	80 bp	0.008612	0.00736846	0.00736846
2y	85 bp	0.009645	0.01242531	0.01242531
3y	90 bp	0.010636	0.01432804	0.01432804

Model and market prices agree to 8 decimals at every bucket, as expected from an exact root-find per step.

Pricing the Bermudan on the calibrated model (PDE grid ±0.087992, i.e. 5×√α(3y) = 5×0.017598): the PDE price is 0.01492071; the LSMC cross-check (10,000 paths, seed 1234) gives 0.01506537 — a 0.97% relative difference, consistent with Monte Carlo sampling noise on what the code itself calls a rough secondary check, not a discrepancy in the underlying model.

Limitations and possible extensions

- **Single-curve**: no OIS/Libor basis distinction.
- **One factor only**: no correlation structure between rates; a 2-factor extension would need a joint volatility calibration and a higher-dimensional PDE or regression.
- **Receiver swaptions**: the Jamshidian branch is not specialized by side, so the project is effectively payer-only.
- **Narrow calibration target**: one σ bucket per Bermudan exercise date on coterminal swaptions only — no smile, no cap/floor cross-calibration, no joint optimizer.
- **LSMC discounting**: uses a flat short-rate proxy between exercise dates rather than a full stochastic discount factor; documented as a secondary sanity check, not the primary engine.
- **No automated tests**: validation is the PDE-vs-LSMC cross-check in `run_example.py` / `Main.ipynb`.

- Natural next steps: a 2-factor HJM version, smile-consistent calibration, and OIS discounting.